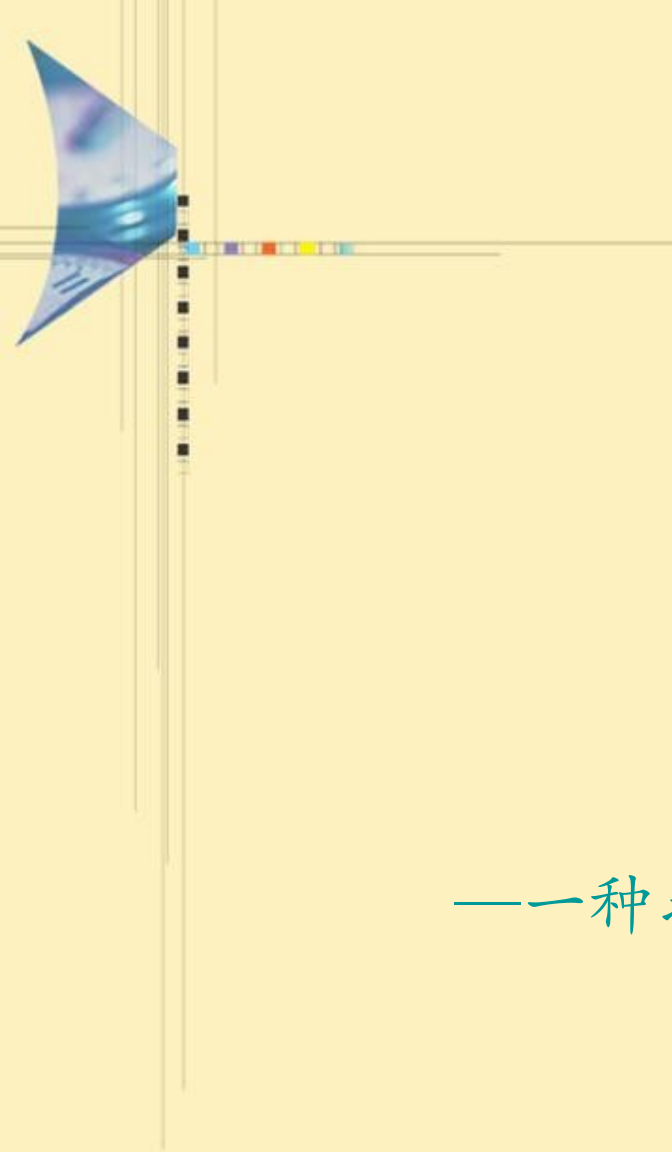




第 5 讲

贪心算法

——一种求解最优化问题的有效算法

主要内容



5.1 引言

5.2 贪心策略的算法设计思想及分析方法

5.3. 贪心策略的经典应用（可选讲部分）





主要内容

5.1 引言

5.1.1 简单问题例的算法描述（最小生成树问题，单源点最短路径问题）

5.1.2 简单问题例算法的分析



5.1 引言

- 顾名思义，**贪心算法**总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。当然，希望贪心算法得到的最终结果也是整体最优的。
- 虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。
- 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是**最优解的很好近似**。

5.1.1 最小耗费生成树——Kruskal 算法



定义 8.1:

- 设 $G = (V, E)$ 是一个具有含权边的连通无向图。 G 的一棵生成树 (V, T) 是 G 的作为树的子图。
- 如给 G 加权并且 T 的各边的权的和为最小值, 那么 (V, T) 就称为最小耗费生成树或简称为最小生成树



问题描述

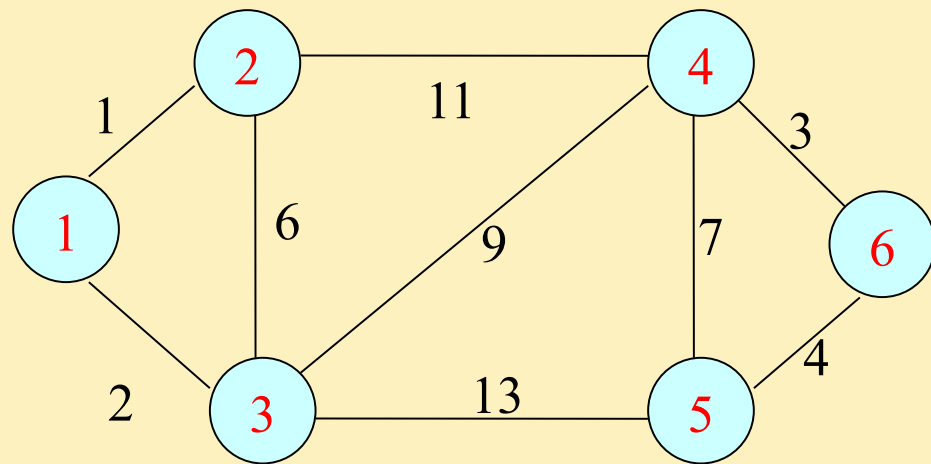
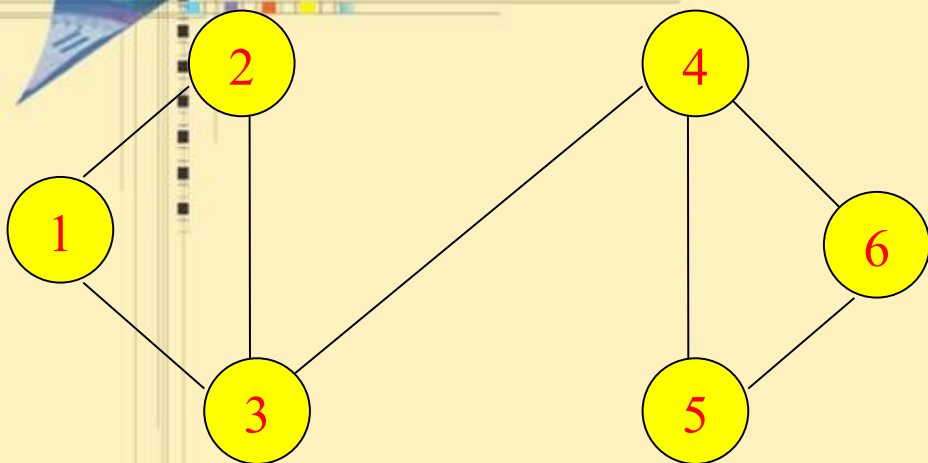
- 假设 G 为连通的, 怎么找到最小生成树 (MST)?



5.1.1 Kruskal 算法基本思想

- 工作原理：维护由许多生成树组成的一个森林，这些生成树逐步合并直到最终森林只由一棵树组成，这棵树就是最小耗费生成树。
- Step 1: 此算法先从对权以非降序排列着手。
- Step 2: 接着从由图的顶点组成而不包含边的森林 (V, T) 开始，重复下面的这一步，直到 (V, T) 被变换成一棵树：
 - 设 (V, T) 是到现在为止构建的森林， $e \in E - T$ 为当前考虑的边，如果把 e 加到 T 中不生成一个回路，那么将 e 加入进 T ，否则丢弃。
 - 这个处理在恰好加完 $n-1$ 条边后结束。

5.1.1 Kruskal 算法基本思想



(1,2)	(1,3)	(4,6)	(5,6)	(2,3)	(4,5)	(3,4)	(2,4)	(3,5)
1	2	3	4	6	7	9	11	13

! 构成回路，这条边被丢弃。

Success!

5.1.1 Kruskal 算法的执行

- 数据结构来表示森林：
 - 为有效地实现此算法，我们需要某种机制来检测加入边后是否构成回路。让它在算法的每个时刻来表示森林，并且在向 T 中添加边时动态检测是否有回路生成。
 - 这种数据结构的一个合适选择是 4.3 节讨论过的不相交集表示法
 - 开始时，图的每个顶点由一棵包含一个顶点的树表示
 - 在算法的执行过程中，森林中的每个连通分量由一棵树来表示。

5.1.1 Kruskal 算法——伪代码

算法 5.1.1 KRUSKAL

输入：包含 n 个顶点的含权连通无向图 G

输出：由 G 生成的最小耗费生成树 T 组成

1. 按非降序权重将 E 中的边排序

2. **for** 每条边 $v \in V$

3. **MAKESET**{ v }

4. **end for**

5. $T = \{\}$

6. **while** $|T| < n-1$

7. 令 (x, y) 为 E 中的下一条边.

8. **if** $\text{FIND}(x) \neq \text{FIND}(y)$ **then** { 检查 (x, y) 是否在一颗树中 }

9. 将 (x, y) 加入 T

10 **UNION**(x, y)

11 **end if**

12. **end while**

➤ 第 1 步和第 2 步分别花费 $O(m \log m)$ 和 $\Theta(n)$, 这里 $m = |E|$.

➤ 第 7 步执行 $n-1$ 次总共要 $\Theta(n)$ 时间。

➤ 合并 **UNION** 运算执行 $n-1$ 次, 查找 **FIND** 运算最多 $2m$ 次.

➤ 第 5 步花费 $\Theta(1)$, 再加上第 9 步最多执行 m 次, 它的总花费是 $O(m)$ 。

➤ 由定理 4.3, 这两个运算的总花费是 $\Theta(m \log n)$ 。

5.1.1 Kruskal 算法——正确性证明

 引理 8.2: 在含权无向图中, 算法 KRUSKAL 正确地找出最小生成树。

• 反证法: 假设 KRUSKAL 生成的不是最小生成树

- 1). 设 KRUSKAL 生成的树为 G_0
- 2). 假设存在 $G_{\min}$ 使得 $\text{cost}(G_{\min}) < \text{cost}(G_0)$ 则在 $G_{\min}$ 中存在 $\langle u, v \rangle$ 不属于 G_0
- 3). 将 $\langle u, v \rangle$ 加入 G_0 中可得一个环, 且 $\langle u, v \rangle$ 不是该环的最长边 (这是因为 $\langle u, v \rangle \in G_{\min}$)
- 4). 这与 KRUSKAL 每次生成最短边矛盾
- 5). 故假设不成立, 命题得证.

Proof P241(or P153)

5.1.1 Kruskal 算法——算法分析

- 第 1 步和第 2 步分别花费 $O(m \log m)$ 和 $\Theta(n)$, 这里 $m = |E|$.
- 第 7 步恰好执行 $n-1$ 次, 总共要 $\Theta(n)$ 时间。合并运算执行 $n-1$ 次, 查找运算最多 $2m$ 次。
- 第 5 步花费 $\Theta(1)$, 再加上第 9 步最多执行 m 次, 它的总花费是 $O(m)$ 。由定理 4.3, 这两个运算的总花费是 $O(m \log^* n)$ 。
- 这样算法总的运行时间取决于排序步, 也就是 $O(m \log m)$
- p153

5.1.1 最小耗费生成树 —— Prim 算法

- 基本思想
- 设 $G=(V,E)$ ，为了简便起见， V 取整数集合 $\{1,2,\dots,n\}$.
- 算法从建立两个顶点集合开始： $X=\{1\}$ 和 $Y=\{2,\dots,n\}$ 。接着生长一棵生成树，每次一条边。
- 在每一步，它找出一条权重最小的边 (x,y) ，这里 $x \in X$ ， $y \in Y$ 并且把 y 从 Y 移到 X 。重复这一步直到 Y 为空。

5.1.1 最小耗费生成树—— Prim 算法

此算法概况如下:

1. $T \leftarrow \{\}$; $X \leftarrow \{1\}$; $Y \leftarrow V - \{1\}$

2. while $Y \neq \{\}$

3. 设 (x,y) 是最小权重的边, 其中 $x \in X$ and $y \in Y$

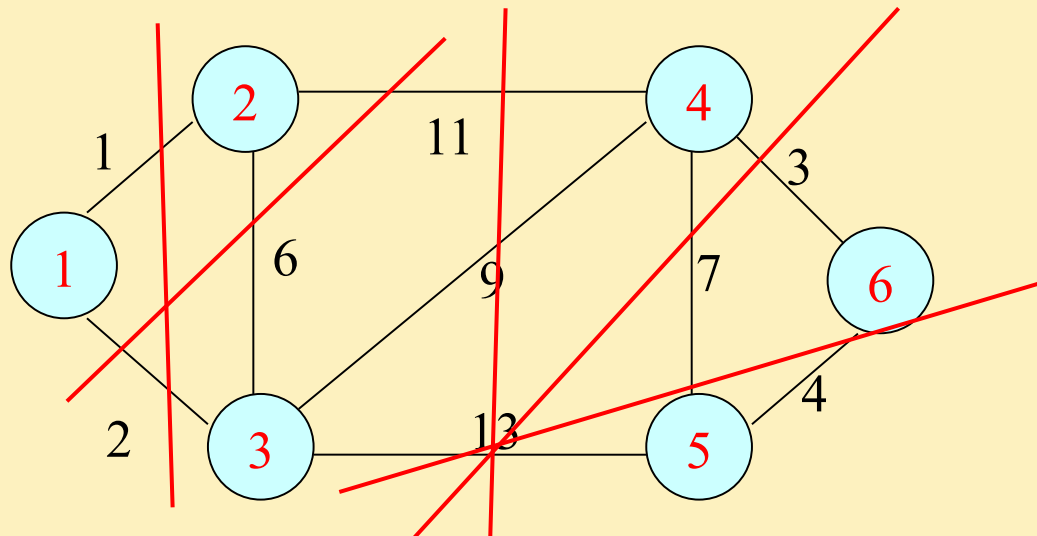
4. $X \leftarrow X \cup \{y\}$

5. $Y \leftarrow Y - \{y\}$

6. $T \leftarrow T \cup \{(x,y)\}$

7. end while

5.1.1 Prim 算法——例子



Success!

5.1.1 Prim 算法——伪代码

算法 PRIM

输入：含权连通无向图 $G=(V,E)$ ，其中 $V=\{1,2,\dots,n\}$ 。

输出：由 G 生成的最小耗费生成树 T 组成的边的集合。

```
1.  $T \leftarrow \{\}$ ;  $X \leftarrow \{1\}$ ;  $Y \leftarrow V - \{1\}$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.     if  $y$  邻接于 1 then
4.          $N[y] \leftarrow 1$ 
5.          $C[y] \leftarrow c[1,y]$ 
6.     else  $C[y] \leftarrow \infty$ 
7.     end if
8. end for
```

➤ 第 1 步耗费 $\Theta(n)$ 时间。

➤ 第 2 步的 for 循环需要时间 $\Theta(n)$ 。

➤ 第 3 步到第 6 步需要时间 $O(n)$ 。

NEXT PAGE

5.1.1 Prim 算法——伪代码

算法 PRIM

```
9.  for j ← 1 to n-1 { 寻找 n-1 条边 }
10.      令 y ∈ Y 使得 C[y] 最小
11.      T ← T ∪ {y, N[y]}
12.      X ← X ∪ {y}
13.      Y ← Y - {y} { 从 Y 中删除 y }
14.      for 每个邻接于 y 的顶点 w ∈ V
15.          if c[y, w] < C[w]
16.              N[w] ← y
17.              C[w] ← c[y, w]
18.          end if
19.      end for
20. end for
```

➤ 第 10 步搜索离 X 最近的顶点 y，每迭代一次需要花费时间 $\Theta(n)$ ，而这一步执行了 n-1 次，所以第 10 步一共花费时间 $\Theta(n^2)$

➤ 第 11 步、第 12 步和第 13 步，每迭代一次花费时间 $\Theta(1)$ 总共花费时间 $\Theta(n)$ 。

➤ 第 14 步的 for 循环执行了 2m 次，第 14 步一共需要的时间是 $\Theta(m)$

5.1.1 Prim 算法——算法分析

- 第 1 步耗费 $\Theta(n)$ 时间。
- 第 2 步的 for 循环需要时间 $\Theta(n)$ 。第 3 步到第 6 步需要时间 $O(n)$ 。
- 第 10 步搜索离 X 最近的顶点 y ，每迭代一次需要花费时间 $\Theta(n)$ ，而这一步执行了 $n-1$ 次，所以第 10 步一共花费时间 $\Theta(n^2)$
- 第 11 步、第 12 步和第 13 步，每迭代一次花费时间 $\Theta(1)$ 总共花费时间 $\Theta(n)$ 。
- 第 14 步的 for 循环执行了 $2m$ 次，第 14 步一共需要的时间是 $\Theta(m)$
- ...P156
- 这就得到了算法的时间复杂性是 $\Theta(m+n^2)$

5.1.1 Prim 算法——稠图的线性时间算法

基本思想

- 现在要改进算法 PRIM，就像曾经对算法 DIJKSTRA 所做的那样，目标是把 $m = o(n^2)$ 的那类图的时间复杂性从 $O(n^2)$ 减少到 $O(m \log n)$ 。以后还要进一步改进它，使在稠图情况下，可以在对边数的线性时间内运行。
- 就像在算法 SHORTESTPATH 中那样，基本思想是用 **最小堆** 数据结构（见 4.2 节）来保持边界顶点集，使得可以在 $O(\log n)$ 时间内取得 Y 集中的顶点 y ，这个 y 和 $V-Y$ 集中一个顶点连接的边的耗费是最小的。修改后的算法在算法 MST 中给出。

算法 MST

输入：含权连通无向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

输出：由 G 生成的最小耗费生成树 T 组成的边的集合。假设我们已有一个空堆 H .

1. $T \leftarrow \{\}$; $Y \leftarrow V - \{1\}$
2. **for** $y \leftarrow 2$ **to** n
3. **if** y 邻接于 1 **then**
4. $N[y] \leftarrow 1$
5. $\text{key}(y) \leftarrow c[1,y]$
6. $\text{INSERT}(H,y)$
7. **else** $\text{key}(y) \leftarrow \infty$
8. **end if**
9. **end for**

NEXT PAGE

算法 MST

```
10. for  $j \leftarrow 1$  to  $n-1$                 { 查找  $n-1$  条边 }
11.    $y \leftarrow \text{DELETEMIN}(H)$ 
12.    $T \leftarrow T \cup \{(y, N[y])\}$         { 添加边  $(y, N[y])$  到  $T$  }
13.    $Y \leftarrow Y - \{y\}$                 { 从  $Y$  删除顶点  $y$  }
14.   for 每个邻接于  $y$  的顶点  $w \in Y$ 
15.     if  $c[y, w] < \text{key}(w)$  then
16.        $N[w] \leftarrow y$ 
17.        $\text{key}(w) \leftarrow c[y, w]$ 
18.     end if
19.     if  $w \notin H$  then  $\text{INSERT}(H, w)$ 
20.     else  $\text{SIFTUP}(H, H^{-1}(w))$ 
21.   end for
22. end for
```

5.1.1 Prim 算法——算法分析



定理

- 给出一个含权无向图 G ，算法 MST 在 $O(m \log n)$ 时间内找出最小耗费生成树。如果图是稠密的，即如果对于某个 $\varepsilon > 0$, $m \geq n^{1+\varepsilon}$ ，那么它可以被进一步改进为在 $O(m/\varepsilon)$ 时间内运行。



5.1.1 单源点最短路径问题

? 问题描述：

- 设 $G = (V, E)$ 是一个每条边有非负长度的有向图，有一个特异顶点 s 称为源。
- 单源最短路径问题，或者简称为最短路径问题，是要确定从 s 到 V 中每一个其他顶点的距离，这里从顶点 s 到顶点 x 的距离定义为从 s 到 x 的最短路径的长度。

5.1.1 单源点最短路径问题——Dijkstra 算法基本思想

贪心技术 : Select the path in nonincreasing order.

- 为简便起见，我们假设 $V=\{1,2,\dots,n\}$, $s=1$.
- 初始时，将顶点分为两个集合 $X=\{1\}$ 和 $Y=\{2,3,\dots,n\}$ 。这样做的目的是让 X 包含这样的顶点集合：从源点到这些顶点的距离已经确定。
- 在每一步中，我们选定源点到它的距离已经获得的一个顶点 $y \in Y$ ，并将这个顶点移到 X 中。
- 与 Y 中的每个顶点 y 联系的是标记 $\square[y]$ ，它是只经过 X 中顶点的最短路径的长度，一旦顶点 $y \in Y$ 移到 X 中，与 y 相邻的每个顶点 $w \in Y$ 的标记就被更新，这表示找到了经过 y 到 w 的更短路径。

5.1.1 Dijkstra 算法——伪代码

算法 DIJKSTRA

输入：含权有向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

输出：G 中顶点 1 到其他顶点的距离。

```
1.  $X=\{1\}$ ;  $Y \leftarrow V-\{1\}$ ;  $\lambda[1] \leftarrow 0$ 
2. for  $y \leftarrow 2$  to  $n$ 
3.     if  $y$  相邻于 1 then  $\lambda[y] \leftarrow \text{length}[1,y]$ 
4.     else  $\lambda[y] \leftarrow \infty$ 
5.     end if
6. end for
7. for  $j \leftarrow 1$  to  $n-1$ 
8.     令  $y \in Y$  使得  $\lambda[y]$  最小
9.      $X \leftarrow X \cup \{y\}$            { 将顶点  $y$  加入  $X$  }
10.     $Y \leftarrow Y - \{y\}$          { 将顶点  $y$  从  $Y$  中删除 }
11.    for 每条边  $(y,w)$ 
12.        if  $w \in Y$  and  $\lambda[y] + \text{length}[y,w] < \lambda[w]$  then
13.             $\lambda[w] \leftarrow \lambda[y] + \text{length}[y,w]$ 
14.        end for
15. end for
```

第 1 步： $\Theta(n)$ 时间。

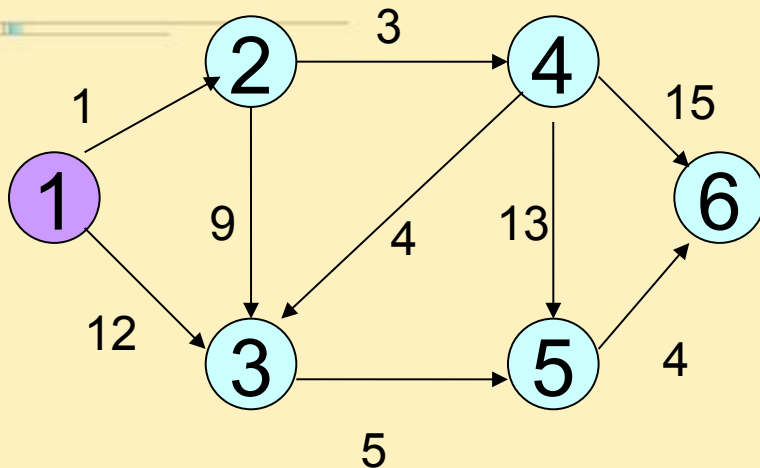
第 3 步 和第 4 步：分别为 $\Theta(n)$ 和 $O(n)$

第 8 步： $\Theta(n^2)$ 。

第 9 步 和第 10 步：每次迭代花费时间 $\Theta(1)$ ，共用了 $\Theta(n)$ 时间。

第 11 步 和第 12 步共需要时间 $\Theta(m)$ 。这里 $m=|E|$ 。

5.1.1 Dijkstra 算法——例子



Y

2	3	4	5	6
1	12	∞	∞	∞

3	4	5	6
10	4	∞	∞

3	5	6
8	17	19

5	6
13	19

6
17

X

{1}

{1,2}

{1,2,4}

{1,2,4,3}

{1,2,4,3,5}

{1,2,4,3,5,6}

Success!



5.1.1 Dijkstra 算法——正确性证明

引理 8.1:

- 在算法 DIJKSTRA 中，当顶点 y 在第 8 步中被选中，如果标记 $\lambda[y]$ 是有限的，那么、 $\lambda[y] = \delta[y]$ 。（ $\delta[y]$ 表示从源点 x 到 y 的距离）
- 证明：
 - 对顶点离开集合 Y 并进入 X 的次序做归纳。
 - 第 1 个离开的顶点是 1，我们有 $\lambda[1] = \delta[1] = 0$ 。假设引理中的结论对于所有在 y 前离开 Y 的顶点都成立。
 - 因为 $\lambda[y]$ 是有限的，则必存在从 1 到 y 的路径，它的长是 $\lambda[y]$ 。设 $\pi = 1, \dots, x, w, \dots, y$ 是从 1 到 y 的最短路径，其中 x 是在 y 前最迟离开 Y 的顶点。我们有 $\lambda[y] \leq \lambda[w] \leq \lambda[x] + \text{length}(x, w) = \delta[x] + \text{length}(x, w) = \delta[w] \leq \delta[y]$ 。
 - 上述的证明基于这样一个假设，即所有的边长都是非负的。
 - p149

5.1.1 Dijkstra 算法——算法分析

- 第 1 步： $\Theta(n)$ 时间。
- 第 3 步和第 4 步：分别为 $\Theta(n)$ 和 $O(n)$
- 第 8 步： $\Theta(n^2)$ 。
- 第 9 步和第 10 步：每次迭代花费时间 $\Theta(1)$ ，共用了 $\Theta(n)$ 时间。
- 第 11 步和第 12 步共需要时间 $\Theta(m)$ 。这里 $m=|E|$ 。
- 根据以上得出算法的时间复杂性是 $\Theta(m+n^2) = \Theta(n^2)$ 。
- p149



定理 8.1

- 给出一个边具有非负权的有向图 G 和源顶点 s ，算法 DIJKSTRA 在时间 $\Theta(n^2)$ 内找出从 s 到其他每一顶点距离的长度

5.1.1 Dijkstra 算法——稠图的线性时间算法



基本思想：

- 用最小堆数据结构来保持集合 Y 中的顶点，使 Y 组中离 $V-Y$ 最近的顶点 y 可以在 $O(\log n)$ 时间内被选出
- 与每个顶点 v 相关的键就是它的标记 $\square[v]$.
- 最后的算法如算法 SHORTESTPATH 所示



算法 SHORTESTPATH

输入：含权有向图 $G=(V,E)$, where $V=\{1,2,\dots,n\}$.

输出：G 中顶点 1 到其他顶点的距离。假设已有一个空堆 H。

1. $Y \leftarrow V - \{1\}$; $\lambda[1] \leftarrow 0$; $\text{key}(1) \leftarrow \lambda[1]$

2. **for** $y \leftarrow 2$ **to** n

3. **if** y 邻接于 1 **then**

4. $\lambda[y] = \text{length}[1,y]$

5. $\text{key}(y) \leftarrow \lambda[y]$

6. $\text{INSERT}(H,y)$

7. **else**

8. $\lambda[y] \leftarrow \infty$

9. $\text{key}[y] \leftarrow \lambda[y]$

10. **end if**

11. **end for**

NEXT PAGE

算法 SHORTESTPATH

```
12. for  $j \leftarrow 1$  to  $n-1$ 
13.      $y \leftarrow \text{DELETEMIN}(H)$ 
14.      $Y \leftarrow Y - \{y\}$  { 将顶点  $y$  从  $Y$  删除 }
15.     for 每个邻接于  $y$  的顶点  $w \in Y$ 
16.         if  $\lambda[y] + \text{length}[y, w] < \lambda[w]$  then
17.              $\lambda[w] \leftarrow \lambda[y] + \text{length}[y, w]$ 
18.              $\text{key}(w) \leftarrow \lambda[w]$ 
19.         end if
20.         if  $w \notin H$  then  $\text{INSERT}(H, w)$ 
21.         else  $\text{SIFTUP}(H, H^{-1}(w))$  {  $H^{-1}(w)$  返回  $w$  在  $H$  中的位置 }
22.         end if
23.     end for
24. end for
```

这里共有 $n-1$ 个 **DELETEMIN** 运算，
 $n-1$ 个 **INSERT** 运算，
最多 $m-n+1$ 个 **SHIFTUP** 运算。
每个堆运算用 $O(\log n)$ 时间，
得到总共需要 $O(m \log n)$ 的时间。

5.1.1 Dijkstra 算法——算法分析

需要强调的：输入算法的是邻接表，如果用邻接矩阵，我们最终会得到一个 $\Theta(n^2)$ 时间的算法。

- 运行时间主要取决于堆运算。
- 这里共有 $n-1$ 个 **DELETEMIN** 运算， $n-1$ 个 **INSERT** 运算，最多 $m-n+1$ 个 **SHIFTUP** 运算。每个堆运算用 $O(\log n)$ 时间，得到总共需要 $O(m \log n)$ 的时间。



定理 8.2

- 给出具有非负权重的边和源顶点 s 的图 G ，算法 **SHORTESTPATH** 可在 **$O(m \log n)$** 时间内找出从 s 到其他每一个顶点的距离。
- 如果图是稠密的，即对于某个 $\epsilon > 0$ ， $m \geq n^{1+\epsilon}$ ，那么它可以被改善为在时间 **time $O(m/\epsilon)$** 内运行。P151



5.1.1 Floyd 与 Dijkstra 算法比较

- Dijkstra 算法, 图所有边权值都为非负的; Floyd 算法, 不允许所有权值为负的回路,
- Dijkstra 只可以求出任意点到达源点的最短距离; Floyd 可以求出任意两点间的最短距离,
- Dijkstra 算法的思想是贪心, Floyd 算法的思想是动态规划
- Dijkstra 时间复杂度 $O(n^2)$, Floyd 算法 $O(n^3)$
- Dijkstra 算法在网络中用得更多, 一个一个节点添加, 加一个点刷一次路由表。Floyd 算法把所有已经连接的路径都标出来, 再通过不等式比较来更改路径。



主要内容

5.2 贪心策略的算法设计思想及分析方法

5.2.1 可行解、部分可行解、贪心解与最优解

5.2.2 贪心准则 / 贪心度量

5.2.3 贪心算法一般框架及特点分析

5.2.4 贪心准则的重要性（分数背包问题为例）

5.2.5 贪心解为最优解的正确性证明（分数背包问题为例）

5.2.6 贪心策略和动态规划策略的异同点



5.2.1 可行解、部分可行解、贪心解与最优解



5.2.2 贪心准则 / 贪心度量

- 贪心算法在少量计算的基础上做出正确猜想而不急于考虑以后的情况。
- 它一步步地来构筑解。每一步均是建立在局部最优解的基础之上，而每一步又都扩大了部分解的规模。
- 做出的选择产生最大的直接收益而又保持可行性。
- 因为每一步的工作很少且基于少量信息，所得算法特别有效。
- 算法由一个简单的迭代过程构成，在维持可行性的前提下它选择能产生最大直接利益的项。
- 贪心算法产生全局最优解的最重要因素是选择策略。

5.2.3 贪心算法一般框架及特点分析

Algorithm GREEDY(A,n)

 solution $\leftarrow \emptyset$

 for $i \leftarrow 1$ to n do

$x \leftarrow \text{SELECT}(A)$

 if FEASIBLE(solution,x)

 then solution $\leftarrow \text{UNION}(\text{solution},x)$

 end if

 end for

 return (solution)





贪心算法特点

- 对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。
- 但是，从许多可用贪心算法求解的问题中我们看到这类问题一般具有 2 个重要的性质：**贪心选择性**和**最优子结构性**。

贪心选择性质

- 所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。
- 动态规划算法通常以自底向上的方式解各子问题，而贪心算法则通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。
- 对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

最优子结构性质

- 当一个问题最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。
- 贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。但是，对于具有最优子结构的问题应该选用贪心算法还是动态规划算法求解？是否能用动态规划算法求解的问题也能用贪心算法求解？

5.2.4 贪心准则的重要性——分数背包问题

? 问题描述：

- 给出 n 个大小为 $s_1, s_2, \dots, s_n$, 值为 $v_1, v_2, \dots, v_n$ 的项目, 并设背包容量为 C , 要找到非负实数 $x_1, x_2, \dots, x_n$ 使和

$$\sum_{i=1}^n x_i v_i$$

- 在约束

$$\sum_{i=1}^n x_i s_i \leq C$$

- 下最大

5.2.4 贪心准则的重要性——例子

- $n=3, C=20, (v_1, v_2, v_3)=(25, 24, 15),$
 $(s_1, s_2, s_3)=(18, 15, 10)$

- 可行方案：

(x_1, x_2, x_3)	$\sum s_i x_i$	$\sum v_i x_i$
(1)(1/2, 1/3, 1/4)	16.5	24.5
(2)(1, 2/15, 0)	20	28.2
(3)(0, 2/3, 1)	20	31
(4)(0, 1, 1/2)	20	31.5

选择策略 -1

- 从剩下的物品中选择最大价值的物品装入背包，得到：
- 方案 (2)(1,2/15,0) 20 28.2 (非最优)
- 原因：虽然价值很大，但是容量的消耗太快

选择策略 -2

- 从剩下的物品中选择最小尺寸的物品装入背包。
- 方案 (3) $(0, 2/3, 1)$ 20 31 (非最优)
- 原因：虽然容量损耗较少，但是价值增长速度太慢

选择策略 -3

- 从剩下的物品中选择最大 v_i/s_i 比率 的物品装入背包。
- Solution (4) (0,1,1/2) 20 31.5 (最优)

5.2.4 贪心准则的重要性——分数背包问题

- Step 1:
 - 每项计算 $y_i = v_i/s_i$, 即该项值和大小的比
- Step 2:
 - 再按比值的降序来排序, 从第一项开始装背包, 然后是第二项, 依次类推, 尽可能地多放, 直至装满背包。

算法 Greedy-KNAPSACK

输入：按照 $v(i)/s(i)$ 比率降序排列的 n 个物品的容量数组 $s[1..n]$ 和物品价值数组 $v[1..n]$ ；背包的总容量 C ，物品的数量 n

输出：最优贪心算法结果 $x[1..n]$

```
for  $i \leftarrow 0$  to  $n$  { 初始化  $x[i]$  }
```

```
     $x[i] \leftarrow 0$ 
```

```
end for
```

```
 $cu \leftarrow C$ 
```

```
for  $i \leftarrow 0$  to  $n$  { 物品按次序装入背包 }
```

```
    if  $s[i] > cu$  then
```

```
        exit
```

```
    end if
```

```
     $x(i) \leftarrow 1$ 
```

```
     $cu \leftarrow cu - s(i)$  { 剩余背包容量 }
```

```
end for
```

```
if  $i \leq n$  then  $x(i) \leftarrow cu/s(i)$  end if { 最后剩余背包容量中装入物品  $i$  的比率 }
```

```
return  $x$ 
```

5.2.5 贪心解为最优解的正确性证明

- 贪心算法最难的部分从不在于问题的求解，而在于正确性的证明，常用的证明方法有归纳法和交换论证法。
- 归纳法：对算法进行步数归纳或问题规模归纳
- 交换论证法：从最优解出发，在保证最优性不变的前提下，从一个最优解进行逐步替换，从而得到贪心策略的解

5.2.5 贪心解为最优解的正确性证明

- 最优装载问题：
 - n 个集装箱 $1, 2, \dots, n$ 装上轮船，集装箱 i 的重量为 w_i ，轮船载重量限制为 c ，无体积限制。如何使装上船的集装箱最多。（假设每个集装箱重量小于 c ）
- 贪心策略：将集装箱按照从轻到重排序，轻者先装
- 正确性证明：对规模归纳
 - 设箱子标号按照从轻到重记为 $1, 2, \dots, n$
 - $n = 1$ 贪心选择显然得到最优解
 - 假设对规模 $n-1$ 的输入得到最优解，证明对规模 n 的输入也得到最优解

5.2.6 贪心策略和动态规划策略的异同点

- 与动态规划算法比较：
 - 贪心算法通常用于求解最优化问题，即量的最大化或最小化。
 - 然而，贪心算法不像动态规划算法，它通常包含一个用以寻找局部最优解的迭代过程。在某些实例中，这些局部最优解转变成了全局最优解，而在另外一些情况下，则无法找到最优解。





主要内容

5.1 贪心策略的经典应用（可选讲部分）

5.3.1 货郎担问题的算法描述、算法分析、正确性证明、程序实现

5.3.2 最优归并模式问题的算法描述、算法分析、正确性证明、程序实现

5.3.3 多段图问题的算法描述、算法分析、正确性证明、程序实现

5.3.4 带期限作业排序问题的算法描述、算法分析、正确性证明、程序实现

5.3.5 霍夫曼编码树问题的算法描述、算法分析、正确性证明、程序实现

5.3.6 活动选择问题的算法描述、算法分析、正确性证明、程序实现

5.3.7 集合覆盖问题的算法描述、算法分析、程序实现（贪心解不一定是最优解）

5.3.1 货郎担问题

- 货郎担问题也叫旅行商问题，即 TSP 问题（Traveling Salesman Problem），是数学领域中著名问题之一。
- **问题描述：**某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员选择出发的城市及旅行路线，使每个城市经过一次，最后回到原出发城市，而总路程最短。
- 相当于求距离最短的哈密尔顿回路，需求从各城市出发的最短回路，再对 n 条回路求极小值。
- 回路最短——贪心选择距离短的路线

5.3.1 货郎担问题

- 例：假设 5 个城市的费用矩阵为：

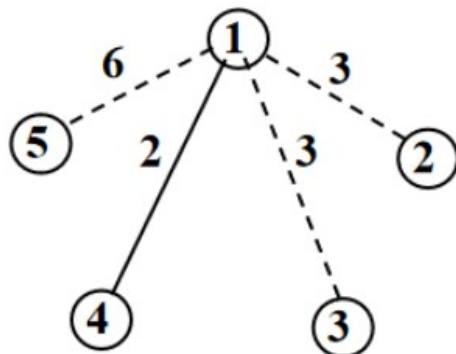
	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

- 从城市 1 出发：1->4->3->5->2->1，总距离 14
- 从城市 2 出发：2->5->4->1->3->2，总距离 17
- 其它城市同理，再在 5 条回路中选择最小距离。

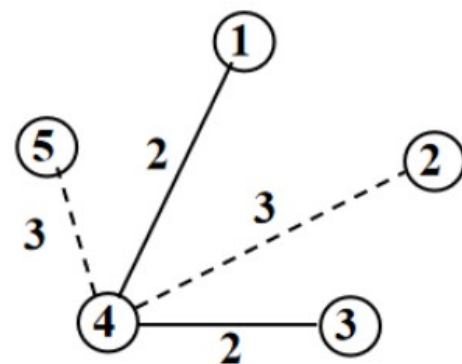
5.3.1 货郎担问题——最近邻点贪心策略

$$C = \begin{pmatrix} \infty & 3 & 3 & 2 & 6 \\ 3 & \infty & 7 & 3 & 2 \\ 3 & 7 & \infty & 2 & 5 \\ 2 & 3 & 2 & \infty & 3 \\ 6 & 2 & 5 & 3 & \infty \end{pmatrix}$$

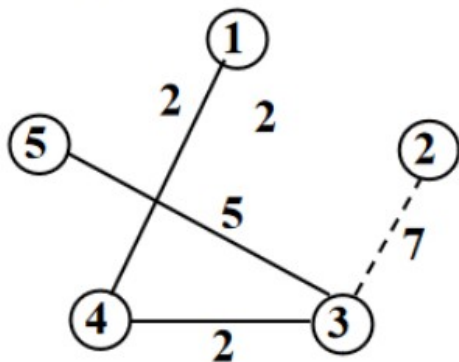
(a) 5城市的代价矩阵



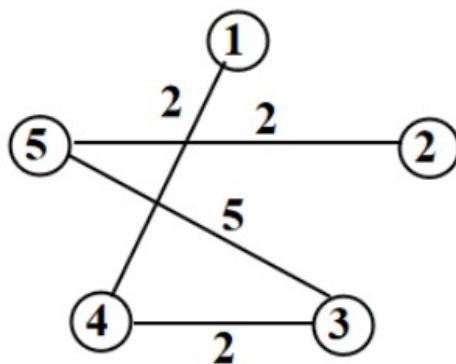
(b) 城市1→城市4



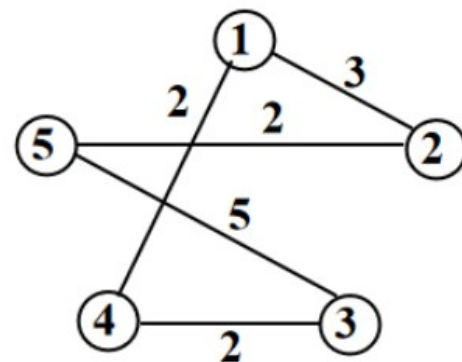
(c) 城市4→城市3



(d) 城市3→城市5



(e) 城市5→城市2



(f) 城市2→城市1

最近邻点贪心策略求解TSP问题的过程，求出的最短路径为1-4-3-5-2-1，长度为14

5.3.1 货郎担问题

	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞

- 从城市 1 出发 :1->4->3->5->2->1 , 总距离 14
- 从城市 2 出发 :2->5->4->1->3->2 , 总距离 17
- 其它城市同理, 再在 5 条回路中选择最小距离。

5.3.1 货郎担问题——时间复杂度

- 时间复杂度分析：
- 从一个城市出发的最短回路计算为 $O(n^2)$ ，共 n 个城市，故时间复杂度为 $O(n^3)$ ，远小于穷举法的 $n!$ 。

5.3.1 货郎担问题——最优解分析

- 最优解分析可得：

从城市 1 出发最优路线 :1->2->5->4->3->1 , 距离 13

- 贪心法求解货郎担问题不具有贪心选择性生质和最优子结构性质。只能得到近似解，不能得到最优解。

5.3.2 最优归并模式问题

- 假设两个分别包含 n 个和 m 个记录的已分类文件可以在 $O(n+m)$ 时间内归并在一起而得到一个分类文件。当要把两个以上的已分类文件归并在一起时，可以通过成对地重复归并已分类的文件来完成。
- 例如，假定 x_1 , x_2 , x_3 , x_4 是要归并的文件，则可以首先把 x_1 和 x_2 归并成文件 y_1 , 然后将 y_1 和 x_3 归并成 y_2 , 最后将 y_2 和 x_4 归并，从而得到想要的分类文件；也可以先把 x_1 和 x_2 归并成 y_1 , 然后把 x_3 和 x_4 归并成 y_2 , 最后归并 y_1 和 y_2 而得到想要的分类文件。

5.3.2 最优归并模式问题——问题描述

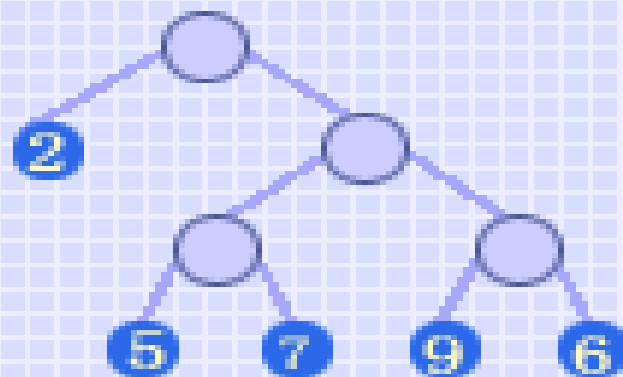
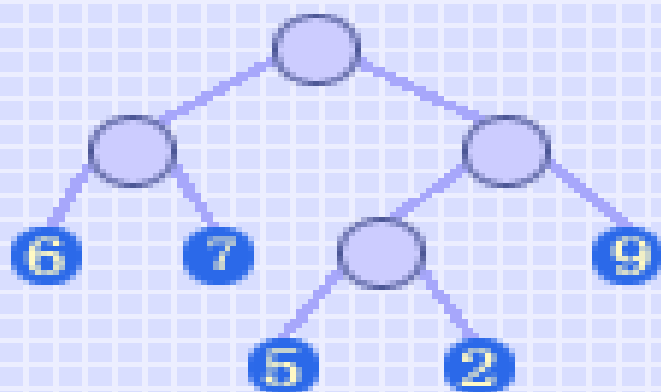
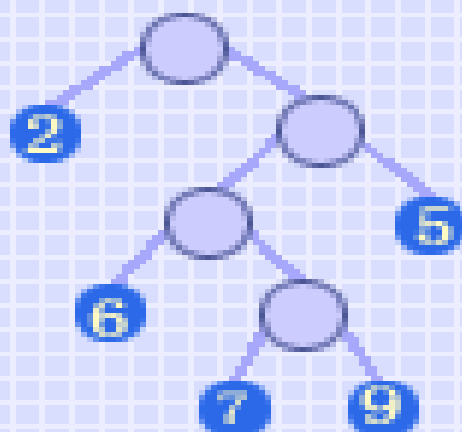
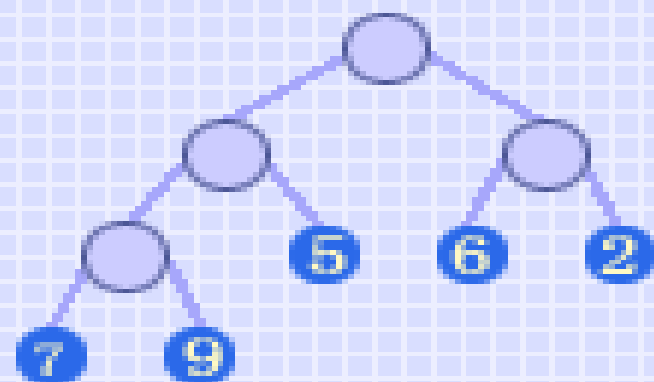
- **问题描述：** 给出 n 个文件，则有许许多多种把这些文件成对地归并成一个单一分类文件的方法。不同的配对法要求不同的计算时间。现在所要论及的问题是确定一个把 n 个已分类文件归并在一起的最优方法（即需要最少比较的方法）。

赫夫曼树及其应用

- **赫夫曼树**：带权路径长度最短的树称为赫夫曼树或最优树。
- **路径**：树的根结点到其余结点的分支。
- **结点的路径长度**：从根结点到该结点的路径上分支的数目。
- **结点的带权路径长度**：从树根到该结点之间的路径长度与该结点上所带权值的乘积。
- **树的带权路径长度**：树中所有叶子结点的带权路径长度之和。

$$WPL = \sum_{k=1}^n w_k l_k$$

- 在所有含 n 个叶子结点、并带相同权值的 m 叉树中，必存在一棵其带权路径长度取最小值的树，称为“**最优树**”



$$WPL = 7 \times 3 + 9 \times 3 + 5 \times 2 + 6 \times 2 + 2 \times 2 = 74 \quad (\text{左上图})$$

$$WPL = 2 \times 1 + 6 \times 3 + 7 \times 4 + 9 \times 4 + 5 \times 2 = 94 \quad (\text{右上图})$$

$$WPL = 6 \times 2 + 7 \times 2 + 5 \times 3 + 2 \times 3 + 9 \times 2 = 65 \quad (\text{左下图})$$

$$WPL = 2 \times 1 + 5 \times 3 + 7 \times 3 + 9 \times 3 + 6 \times 3 = 83 \quad (\text{右下图})$$

赫夫曼树的构造

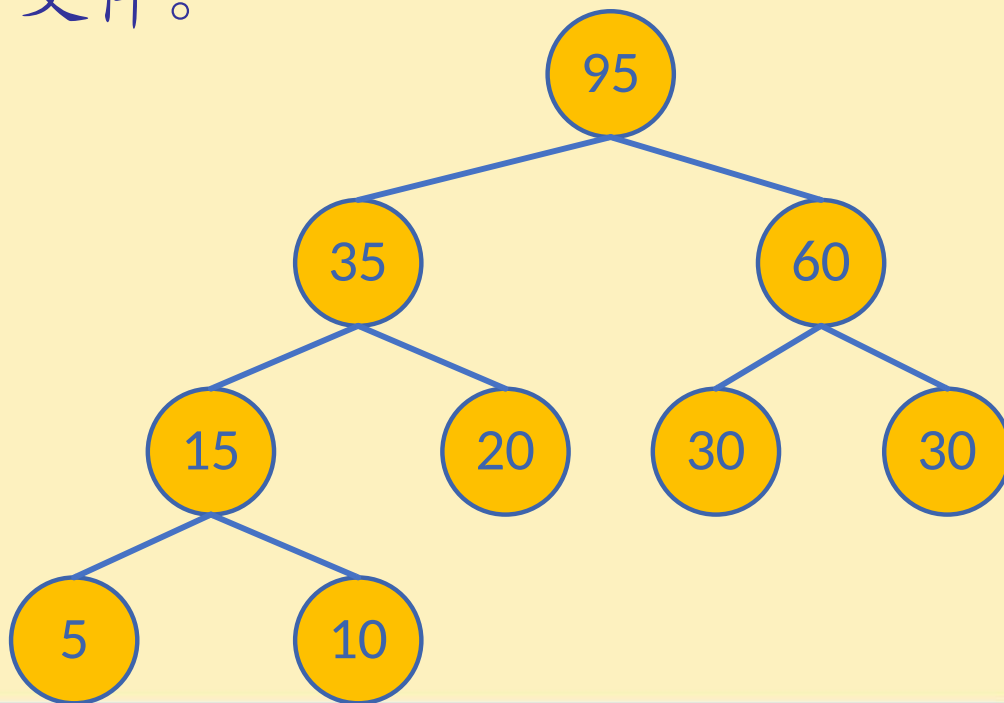
- (1) 根据给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ ，构造 n 棵二叉树的集合 $F = \{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树中均只含一个带权值为 w_i 的根结点，其左、右子树为空树；
- (2) 在 F 中选取其根结点的权值为最小的两棵二叉树，分别作为左、右子树构造一棵新的二叉树，并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和；（根结点的权值 = 左右孩子权值之和，叶结点的权值 = w_i ）。
- (3) 从 F 中删去这两棵树，同时加入刚生成的新树；
- (4) 重复 (2) 和 (3) 两步，直至 F 中只含一棵树为止。

5.3.2 最优归并模式问题——最优度量标准

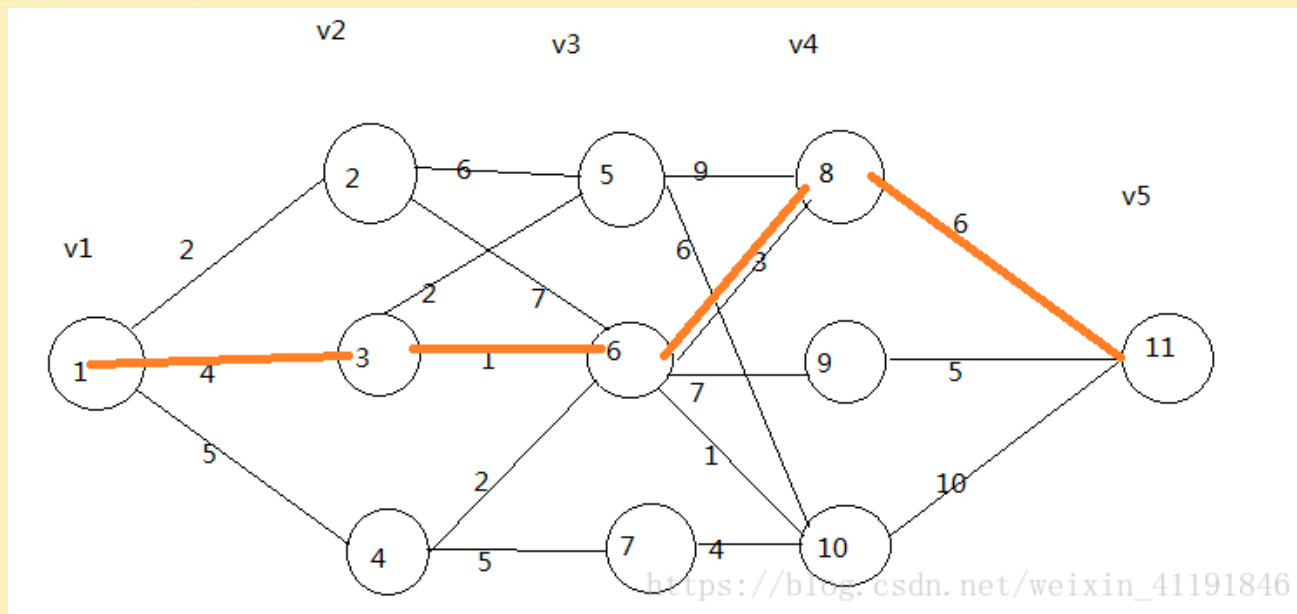
- 最优归并模式：赫夫曼树
- 最优度量标准：带权路径长度最小

5.3.2 最优归并模式问题——例子

- 例：将 5 个长度分别为 {20, 30, 30, 10, 5} 的有序文件 (F1, F2, F3, F4, F5) 两两合并成一个有序文件。



5.3.3 多段图问题



- 按照贪心算法的思路解决问题可以得到：v1 到 v2：1 到 2 最短，所以选择 2，总长 s 为 2。v2 到 v3：2 到 5 最短，所以选择 5，总长为 $2+6=8$ 。v3 到 v4：5 到 10 最短，所以选择 10，总长为 $8+6=14$ 。v4 到 v5：10 到 11，总长为 $14+10=24$ 。所以最短路径为 1,2,5,10,11 但是这明显不是最短路径，所以通过贪心算法思路解决的话是错误的。

5.3.4 带期限作业排序问题——问题描述

问题描述:

- 带有期限的作业排序要解决的是操作系统中单机、无资源约束且每个作业可在等量的时间内完成的作业调度问题。把这个问题形式化描述为:
- ①要在一台机器上处理 n 个作业, 每个作业可以在单位时间内完成;
- ②每个作业 i 都有一个期限值 d_i , $d_i > 0$;
- ③当作业在它规定的期限值前完成, 就可以获得的效益 p_i , $p_i > 0$;

5.3.4 带期限作业排序问题——求解目标

- 问题求解的目标是：问题的可行解是这 n 个作业的一个子集合 J 。 J 中的每个作业都能在各自的截止期限前完成后，产生一个作业效益之和。
- 我们的目标就是找到一个子集 J ， J 中的每个作业都能在各自的截止期限前完成，并且使得作业效益值的和最大。这个作业的一个子集合 J 就是所求的最优解。

5.3.4 带期限作业排序问题——例子

- 例 $n=4$, $(p_1,p_2,p_3,p_4)=(100,10,15,20)$,
 $(d_1,d_2,d_3,d_4)=(2,1,2,1)$ 。这个问题的最优解为第 7 个,
所允许的处理顺序是: 先处理作业 4 , 再处理作业 1 。
在时间 0 开始处理作业 4 , 而在时间 2 完成对作业 1 的
处理。可行解 处理顺序 效益

1	{1}	1	100
2	{2}	2	10
3	{3}	3	15
4	{4}	4	20
5	{1 , 2}	2 , 1	110
6	{1 , 3}	1 , 3 或 3 , 1	115
7	{1 , 4}	4 , 1	120
8	{2 , 3}	2 , 3	25
9	{3 , 4}	4 , 3	35

5.3.4 带期限作业排序问题——贪心度量标准

- 贪心算法度量标准的选取：目标是作业效益值的和最大
- 因为 $(p_1, p_2, p_3, p_4) = (100, 10, 15, 20)$
- 把作业按照它们的效益值作一个非增次序的排序。作业根据效益值排序为：P1、P4、P3、P2。

5.3.4 带期限作业排序问题——贪心度量标准

- 求解时首先把作业 1 计入 J ，由于作业 1 的期限值 $d_1=2$ ，所以 $J=\{1\}$ 是一个可行解。接下来计入作业 4。由于作业 4 的期限值 $d_4=1$ 而作业 1 的期限值 $d_1=2$ ，可以先完成作业 4 后再完成作业 1，所以 $J=\{1, 4\}$ 是一个可行解。
- 然后考虑作业 3，但是作业 3 的期限值 $d_3=2$ ，计入 J 后就没有办法保证 J 中的每一个作业都在期限值内完成，因此计入作业 3 后不能构成一个可行解。同理计入 2 后也不能构成一个可行解。
- 由分析可知，把目标函数作为度量标准能够得到问题的最优解。

5.3.4 带期限作业排序问题——算法

- procedure GREEDY-JOB(D, J, n)
- // 作业按照 $p_1 \geq p_2 \geq \dots \geq p_n$ 的次序输入，它们的期限值 $D(i) \geq 1, 1 \leq i \leq n, n \geq 1$ 。J 是在它们的截止期限前完成的作业集合。
- $J \leftarrow \{1\}$
- For $i \leftarrow 2$ to n do
- If $J \cup \{i\}$ 的所有作业都能在它们的截止期限前完成
- Then $J \leftarrow J \cup \{i\}$ Endif
- Repeat
- End GREEDY-JOB

5.3.5 霍夫曼编码树

? 问题描述：

- 假设有一个字符串文件，我们希望用这样一种方法尽可能多地压缩文件，但源文件能够很容易地被重建。
- 设文件中的字符集是 $C = \{c_1, c_2, \dots, c_n\}$ ，又设 $f(c_i), 1 \leq i \leq n$ ，是文件中字符 c_i 的**频度**，即文件中 c_i 出现的次数。
- 用**定长**比特数表示每个字符，称为字符的编码，文件的大小取决于文件中的字符数。

5.3.5 霍夫曼编码树——变长编码

- 然而由于有些字符的频度可能远大于另外一些字符的频度，所以用**变长的编码**是有道理的。从直观上说，那些频度大的字符将被赋予短的编码，而长的编码可以赋给那些频度小的字符。
- 当编码在长度上变化时，我们规定一个字符的编码必须不能是另一个字符编码的前缀（即词头），这种码称为**前缀码**。
- 例如，如果我们把编码 10 和 101 赋予字符“a”和“b”就会存在二义性，不清楚 10 究竟是“a”的编码还是字符“b”编码的前缀。

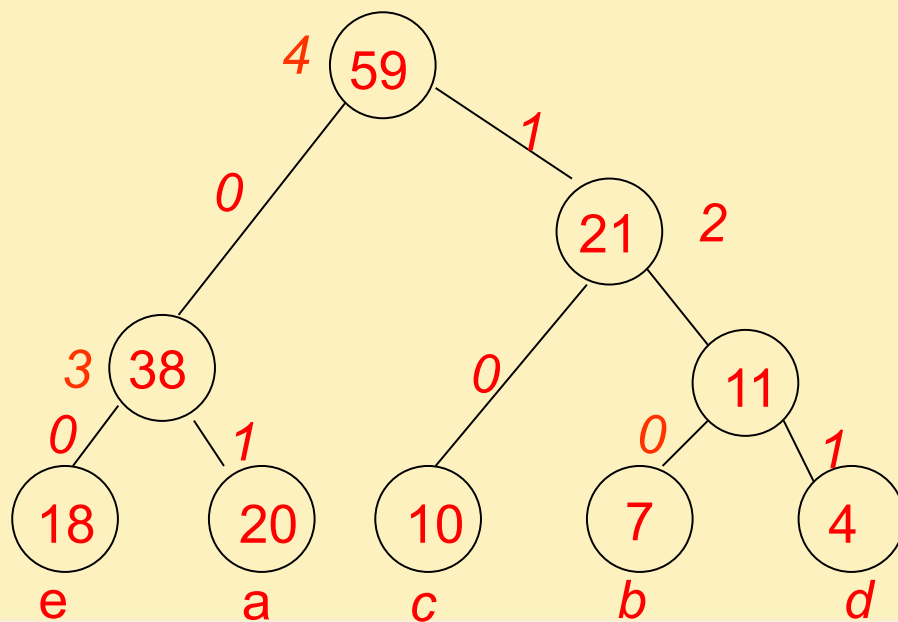
5.3.5 霍夫曼编码树—— Huffman 编码

- 一旦满足前缀约束，编码就变得没有二义性了，就可以扫描比特序列直到找到某个字符的编码。
- 对给定比特序列进行分析的一种方法是用一棵满二叉树，它的每一个内部节点都恰好有两个分支，标记为 0 和 1，这棵树中的叶子对应于字符，从根到叶的每一条路径上的 0 和 1 的序列对应于一个字符的编码。
- Huffman 编码：
 - 这个算法由 Huffman 提出。由算法构造的编码满足前缀约束，并且最小化压缩文件的大小。

5.3.5 霍夫曼编码树——例子

- 文件由字母 a, b, c, d 和 e 组成, 频度为:

$$f(a) = 20, f(b) = 7, f(c) = 10, f(d) = 4 \text{ and } f(e) = 18$$



算法 HUFFMAN

输入：n 个字符的集合 $C=\{c_1, c_2, \dots, c_n\}$ 和它们的频度 $\{f(c_1), f(c_2), \dots, f(c_n)\}$.

输出：C 的 Huffman 树 (V, T)

1. 根据频度将所有字符插入最小堆 H
2. $V \leftarrow C; T = \{\}$
3. **for** $j \leftarrow 1$ **to** $n-1$
4. $c \leftarrow \text{DELETEMIN}(H)$
5. $c' \leftarrow \text{DELETEMIN}(H)$
6. $f(v) \leftarrow f(c) + f(c')$ $\{v \text{ 是一个新节点}\}$
7. $\text{INSERT}(H, v)$
8. $V = V \cup \{v\}$ $\{\text{添加 } v \text{ 到 } V\}$
9. $T = T \cup \{(v, c), (v, c')\}$ $\{\text{使 } c \text{ 和 } c' \text{ 成为 } T \text{ 中 } v \text{ 的孩子}\}$
10. **end for**

5.3.5 霍夫曼编码树——算法分析

- 把所有字符插入堆中需要时间 $\Theta(n)$ 。
- 从堆中删除两个元素和加一个新元素需要的时间是 $O(\log n)$ 。
- 因为这被重复 $n-1$ 次，for 循环需要的所有时间是 $O(n \log n)$ ，
- 就得到算法的时间复杂性是 $O(n \log n)$

5.3.6 活动选择问题

【问题描述】 假设有一个需要使用某一资源的 n 个活动所组成的集合 S ， $S=\{1, \dots, n\}$ 。该资源任何时刻只能被一个活动所占用，活动 i 有一个开始时间 b_i 和结束时间 e_i ($b_i < e_i$)，其执行时间为 $e_i - b_i$ ，假设最早活动执行时间为 0。

一旦某个活动开始执行，中间不能被打断，直到其执行完毕。若活动 i 和活动 j 有 $b_i \geq e_j$ 或 $b_j \geq e_i$ ，则称这两个活动兼容。

设计算法求一种最优活动安排方案，使得所有安排的活动个数最多。

5.3.6 活动选择问题

【问题求解】 假设活动时间的参考原点为 0。一个活动 i ($1 \leq i \leq n$) 用一个区间 $[b_i, e_i)$ 表示, 当活动按结束时间 (右端点) 递增排序后, 两个活动 $[b_i, e_i)$ 和 $[b_j, e_j)$ 兼容 (满足 $b_i \geq e_j$ 或 $b_j \geq e_i$) 实际上就是指它们不相交。

用数组 A 存放所有的活动, $A[i].b$ ($1 \leq i \leq n$), 存放活动起始时间, $A[i].e$ 存放活动结束时间。

例如，对于下表的 $n=11$ 个活动（已按结束时间递增排序）

A:

i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15

产生最大兼容活动集合的过程:

活动 1	✓
活动 2	×
活动 3	×
活动 4	✓
活动 5	×
活动 6	×
活动 7	×
活动 8	✓
活动 9	×
活动 10	×
活动 11	✓

最大兼容活动集合:

活动 1 活动 4 活动 8 活动 11

1 4 8

└──────────────────┘

求解结果

// 问题表示

```
struct Action          // 活动的类型声明
{
    int b;              // 活动起始时间
    int e;              // 活动结束时间
    bool operator<(const Action &s) const // 重载 < 关系函数
    {
        return e<=s.e; // 用于按活动结束时间递增排序
    }
};
int n=11;
Action A[]={ {0}, {1,4}, {3,5}, {0,6}, {5,7}, {3,8}, {5,9}, {6,10},
{8,11},
{8,12}, {2,13}, {12,15}}; // 下标 0 不用
```

// 求解结果表示

```
bool flag[MAX]; // 标记选择的活动
int Count=0;     // 选取的兼容活动个数
```

```
void solve()
```

```
{  memset(flag,0,sizeof(flag));
```

```
    sort(A+1,A+n+1);
```

```
    int preend=0;
```

```
    for (int i=1;i<=n;i++)
```

```
    {  if (A[i].b>=preend)
```

```
        {  flag[i]=true;
```

```
            preend=A[i].e;
```

```
        }
```

```
    }
```

```
}
```

```
// 求解最大兼容活动子集
```

```
// 初始化为 false
```

```
//A[1..n] 按活动结束时间递增排序
```

```
// 前一个兼容活动的结束时间
```

```
// 扫描所有活动
```

```
// 找到一个兼容活动
```

```
// 选择 A[i] 活动
```

```
// 更新 preend 值
```

5.3.6 活动选择问题——算法分析

【算法分析】 算法的主要时间花费在排序上，排序时间为 $O(n\log 2n)$ ，所以整个算法的时间复杂度为 $O(n\log 2n)$ 。



5.3.6 活动选择问题——算法证明

【算法证明】 通常证明一个贪心选择得出的解是最优解的一般的方法是，构造一个初始最优解，然后对该解进行修正，使其第一步为一个贪心选择，证明总是存在一个以贪心选择开始的求解方案。

对于本问题，所有活动按结束时间递增排序，就是要证明：若 X 是活动安排问题 A 的最优解， $X = X' \cup \{1\}$ ，则 X' 是 $A' = \{i \in A : e_i \geq b_1\}$ 的活动安排问题的最优解。

5.3.6 活动选择问题——算法证明

首先证明总存在一个以活动 1 开始的最优解。

如果第一个选中的活动为 k ($k \neq 1$)，可以构造另一个最优解 Y ， Y 中的活动是兼容的， Y 与 X 的活动数相同。

那么用活动 1 取代活动 k 得到 Y' ，因为 $e_1 \leq e_k$ ，所以 Y' 中的活动是兼容的，即 Y' 也是最优的，这就说明总存在一个以活动 1 开始的最优解。

5.3.6 活动选择问题

当做出了对活动 1 的贪心选择后，原问题就变成了在活动 2、...、n 中找与活动 1 兼容的那些活动的子问题。亦即，如果 X 为原问题的一个最优解，则 $X' = X - \{1\}$ 也是活动选择问题 $A' = \{i \in A \mid b_i \geq e_1\}$ 的一个最优解。

反证法：如果能找到一个 A' 的含有比 X' 更多活动的解 Y' ，则将活动 1 加入 Y' 后就得到 A 的一个包含比 X 更多活动的解 Y ，这就与 X 是最优解的假设相矛盾。

因此，在每一次贪心选择后，留下的是一个与原问题具有相同形式的最优化问题，即最优子结构性质。

5.3.6 活动选择问题——例子

例：求解蓄栏保留问题。农场有 n 头牛，每头牛会有一个特定的时间区间 $[b, e]$ 在蓄栏里挤牛奶，并且一个蓄栏里任何时刻只能有一头牛挤奶。

现在农场主希望知道最少蓄栏能够满足上述要求，并给出每头牛被安排的方案。对于多种可行方案，输出一种即可。



解：牛的编号为 $1 \sim n$ ，每头牛的挤奶时间相当于一个活动，与前面活动安排问题不同，这里的活动时间是闭区间，例如 $[2, 4]$ 与 $[4, 7]$ 是交叉的，它们不是兼容活动。

采用与求解活动安排问题类似的贪心思路，将所有活动这样排序：结束时间相同按开始时间递增排序，否则按结束时间递增排序。

求出一个最大兼容活动子集，将它们安排在一个蓄栏中（蓄栏编号为 1）；如果没有安排完，再在剩余的活动再求下一个最大兼容活动子集，将它们安排在另一个蓄栏中（蓄栏编号为 2），以此类推。

也就是说，最大兼容活动子集的个数就是最少蓄栏个数。



i	1	2	3	4	5	6	7
开始时间	1	2	5	8	4	12	11
结束时间	4	5	7	9	10	13	15



1	3	4	6	2	7	5
1	5	8	12	2	11	4
4	7	9	13	5	15	10



最大兼容活动子集个数为 3

// 问题表示

```
struct Cow
{
    int no;
    int b;
    int e;
    bool operator<(const Cow &s) const // 重载 < 关系函数
    {
        if (e==s.e) // 结束时间相同按开始时间递增排序
            return b<=s.b;
        else // 否则按结束时间递增排序
            return e<=s.e;
    }
};
int n=5;
Cow A[]={ {0}, {1,1,10}, {2,2,4}, {3,3,6}, {4,5,8}, {5,4,7} };
// 下标 0 不用

// 求解结果表示
int ans[MAX]; //ans[i] 表示第 A[i].no 头牛的蓄栏编号
```

```

void solve()
{
    sort(A+1,A+n+1);
    memset(ans,0,sizeof(ans));
    int num=1;

    for (int i=1;i<=n;i++)
    {
        if (ans[i]==0)
        {
            ans[i]=num;
            int preend=A[i].e;

            for (int j=i+1;j<=n;j++) // 查找一个最大兼容活动子集
            {
                if (A[j].b>preend && ans[j]==0)
                {
                    ans[j]=num; // 将兼容活动子集中活动安排在 num 蓄栏中
                    preend=A[j].e; // 更新结束时间
                }
            }
            num++; // 查找下一个最大兼容活动子集 , num 增 1
        }
    }
}

```

栏

时间

```
void main()
{ solve();
  printf(" 求解结果 \n");
  for (int i=1;i<=n;i++)
    printf("      牛 %d 安排的蓄栏 : %d\n",A[i].no,ans[i]);
}
```



i	1	2	3	4	5
开始时间	1	2	3	5	4
结束时间	10	4	6	8	7



求解结果

牛 2 安排的蓄栏 :1

牛 3 安排的蓄栏 :2

牛 5 安排的蓄栏 :3

牛 4 安排的蓄栏 :1

牛 1 安排的蓄栏 :4



5.3.7 集合覆盖问题

- 集合覆盖问题（Set covering problem, SCP）是组合数学、计算机科学和计算复杂性理论中的一个经典问题。
- 集合覆盖的决定性问题是卡普的二十一个 NP- 完全问题之一。

5.3.7 集合覆盖问题——问题描述

- **问题描述：** 一个集合 U 以及 U 内元素构成的若干个小类集合 S ，目标是找到 S 的一个子集，该子集满足所含元素包含了 U 中的所有元素且使小类集合个数最少。
- 例如， $U=\{1,2,3,4,5\}$ ， $S=\{\{1,2\},\{3,4\},\{2,4,5\},\{4,5\}\}$ ，找到集合能满足条件的可以有 $O=\{\{1,2\},\{3,4\},\{4,5\}\}$ 或是 $O=\{\{1,2\},\{3,4\},\{2,4,5\}\}$ ，

5.3.7 集合覆盖问题——贪心策略

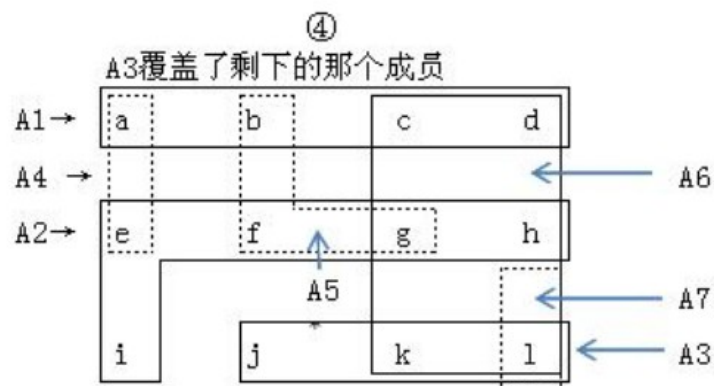
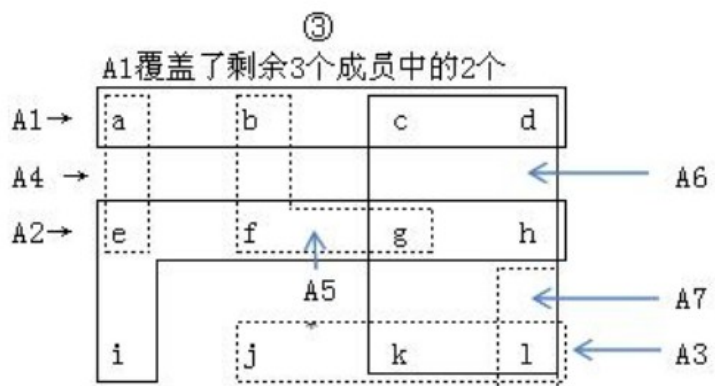
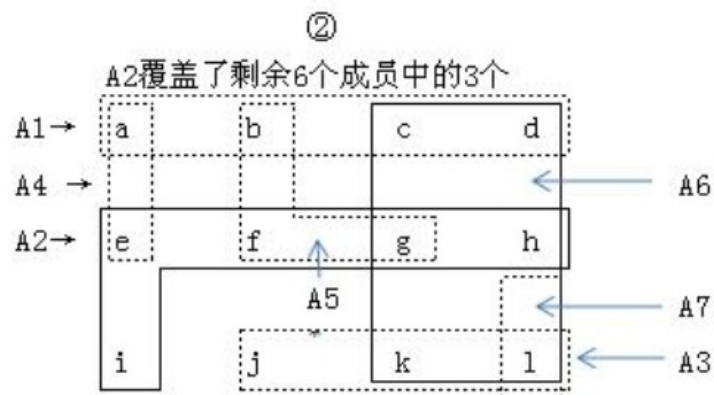
- **方法 1- 蛮力穷举搜索**：最少集合个数 k 从 1 到 m 进行遍历，针对每个子集，遍历 $C(m,k)$ 种集合组合情况。对每种情况求集合并集，观察是否满足全元素覆盖；找到第一个满足条件的情况时结束搜索，输出 k 。
- 特点是可求出确切解 (exact solution)，但是当 k 值较大时组合空间过大且涉及频繁的集合求并效率极低—— n 和 m 较小时可行但较大时不可行，计算时间过长。
- **方法 2- 贪心策略**：每次迭代添加包含未覆盖元素最多的集合，直到满足全覆盖条件。
- 特点是速度快但只是近似解而非确切解。

5.3.7 集合覆盖问题——例子

- 例子：对于 12 种技能的集合 $S=\{a,b,c,d,e,f,g,h,i,j,k,l\}$ 的最佳覆盖集。现在考虑有 7 名待选选手的集合 $P=\{A1,...A7\}$ 。P 中选手拥有的技能集合为：
 $A1=\{a,b,c,d\}$ ， $A2=\{e,f,g,h\}$ ， $A3=\{j,k,l\}$ ，
 $A4=\{a,e\}$ ， $A5=\{b,f,g\}$ ， $A6=\{c,d,g,h,k,l\}$ ， $A7=\{l\}$ 。
- 最佳覆盖集应该是 $C=\{A1,A2,A3\}$ 。这里给出的算法选择的集合是 $C=\{A6,A2,A1,A3\}$ 。

5.3.7 集合覆盖问题——例子

- 最佳覆盖集应该是 $C=\{A1,A2,A3\}$ 。这里给出的算法选择的集合是 $C=\{A6,A2,A1,A3\}$ 。



5.3.7 集合覆盖问题——贪心的近似度分析

Theorem 1. *The greedy algorithm is a $\Theta(\log n)$ approximation.*

u_j 在贪心算法第j次迭代还没有被覆盖的元素个数

$k = \text{OPT}$ = 最少集合个数

k 个最优集合必定可覆盖住该 u_j 个元素, 故最优集合中至少有一个集合包含至少 u_j/k 个元素

$$u_{j+1} \leq \left(u_j - \frac{u_j}{k}\right) \leq \left(1 - \frac{1}{k}\right) u_j \leq \left(1 - \frac{1}{k}\right) \left(1 - \frac{1}{k}\right) u_{j-1} \leq \dots \leq \left(1 - \frac{1}{k}\right)^{j+1} u_0 = \left(1 - \frac{1}{k}\right)^{j+1} n$$

$$\lim_{n \rightarrow \infty} \left(1 + \frac{x}{n}\right)^n = e^x \quad u_{j+1} \leq \left(1 - \frac{1}{k}\right)^{j+1} n$$

$$\left(1 - \frac{1}{k}\right)^i = \left(\left(1 - \frac{1}{k}\right)^k\right)^{\frac{i}{k}} \leq e^{-\frac{i}{k}}$$

$u_i \leq 1$ 迭代终止条件, i 次迭代后, 只差最多1个元素未被覆盖

$$n \cdot e^{-\frac{i}{k}} \leq 1 \iff e^{-\frac{i}{k}} \leq n^{-1} \iff -i/k \leq -\ln n \iff i \geq k \ln n$$

whenever $i \geq k \cdot \ln n$ we have that $u_i \leq 1$

a set cover whose size is at most $k \cdot \ln n + 1 = \text{OPT} \ln n + 1 \in \Theta(\ln n \cdot \text{OPT}) = \Theta(\log n \cdot \text{OPT})$